GANs for Stock Market Prediction

Allison Powell, Varisha Shams, Hari Iyer

(Dated: May 2024)

I. INTRODUCTION

Generative Adversarial Networks (GANs) represent a compelling neural network architecture renowned for their prowess in generating high-fidelity images. The idea was first introduced in 1991 by uergen Schmidhuber in his paper "A Possibility for Implementing Curiosity and Boredom in Model-Building Neural Controllers" [1]. Through iterative training, GANs can uncover complex patterns and dependencies in historical data, potentially leading to more accurate predictions of future market movements and time series forecasting. GANs' pivotal role in advancing the limits of generative modeling piques our interest, since we believe we can utilize its broad applicability to model the predicted prices in the stock market. GANs involve two neural networks: a generator and a discriminator. The generator creates fake data from random noise, to try to fool the discriminator (which distinguishes real from fake). Both networks learn through backpropagation, with the generator minimizing its errors and the discriminator maximizing its ability to accurately distinguish between data. What makes this model interesting is that it allows us to create 'fake' data from mapping vectors to a data space that can serve as a realistic sample for the benefit of our financial applications.

In section II, we descibe the mathematical theory behind GANs. In section III we will apply a cGAN trained on the fashion MNIST dataset to generate clothing images. In section IV we will conduct and investigation of stock price prediction utilizing Generative Adversarial Networks (GANs). In particular, we will discuss the results of the 2021 paper "Stock price prediction using General Adversarial Networks" [2].

II. THEORY

Let X be a set of data distributed according to a distribution P_{data}, and let G and D be neural networks with weights θ_g and θ_d respectively. $P_z(z)$ is a noise distribution. Random data is sampled from the noise distribution and fed into the generator model. The out put of the generator G(z) is distributed according to $P_g(x)$. The output from the generator and the original data are both passed to the discriminator which outputs the probability that the input belongs to the real data. In this way, the discriminator is just a binary classifier. When the original data is input into the discriminator, we will give it the label y = 1. When the generated data is input, we will label it y = 0. D tries to maximize the probability that it will correctly predict the class while G tries to minimize the probability that D will guess correctly. Essentially, the generator, G, tries to create data that mimics the actual data distribution P_{data}, while the discriminator, D, evaluates whether a given data sample looks 'real' (from P_{data}) or 'fake' (from G). The function that D is trying to maximize and G is trying to minimize is the Value function which we will develop in the following discussion.

Recall that the Kullback-Leibler divergence can be decomposed as follows:

$$D_{KL}(P||Q) := \int log(\frac{p(x)}{q(x)})p(x)dx = \int log(p(x))p(x)dx - \int log(q(x))p(x)dx \tag{1}$$

Here, $D_{KL}(P||Q)$ quantifies how one probability distribution P diverges from a second expected probability distribution Q.

The second term corresponds to the cross entropy. For the binary classification problem we are dealing with, we will use the binary cross entropy which can be written as follows:

$$\sum yln(\hat{y}) + (1-y)ln(1-\hat{y}) \tag{2}$$

In this equation, y can be either 0 or 1. $\hat{y}$ is the probability that y will be 1 for all points. Thus, each time y=1, $ln(\hat{y})$ is added to the loss, and $ln(1-\hat{y})$ is added whenever y = 0. For

our purposes, we will construct the cross entropy such that:
when y = 1 $\hat{y}$ = D(x) $\rightarrow L = ln[D(x)]$
when y = 0 $\hat{y}$ = D(G(z)) $\rightarrow L = ln[1 - D(G(z)]$

So we have:

$$L = ln[D(x)] + ln[1 - D(G(z))] \tag{3}$$

This objective captures the essence of the training process for GANs, where D aims to maximize this loss (i.e., correctly classifying real and generated samples), whereas G tries to minimize the second term by fooling D.

This expression applies to only one data point. In order to take into account the rest of the data, we must multiply the loss at each point by the probability at that point and then integrate over the entire distribution. We do this for both the real and generated distributions.

$$V(G, D) = \int P_{data}(x) ln[D(x)]dx + \int P_z(z) ln[1 - D(G(z))]dz \tag{4}$$

This formula expresses the total value function V for a Generative Adversarial Network, where the discriminator D tries to maximize the probability of correctly identifying real and fake data samples.

But this is just the definition of the expectation:

$$V(G, D) = E_{x\ P_{data}}[ln(D(x))] + E_{z\ P_z}[ln(1 - D(G(z))] \tag{5}$$

In practice, the optimization technique we use to play this minimax game is gradient descent. Fixing the learning of G, we can take m data samples and m fake data samples, and then update θ_d by gradient ascent to maximize the value function.

$$\frac{\partial}{\partial \theta_d} \frac{1}{m} [ln[D(x)] + ln[1 - D(G(z))]] \tag{6}$$

The goal is to update θ_d to increase the likelihood of correctly classifying real and generated

samples. Next, we fix the learning of D and then take m fake data samples and update θ_g by gradient descent

$$\frac{\partial}{\partial \theta_g}\frac{1}{m}[ln[1 - D(G(z))]] \tag{7}$$

This equation adjusts the generator's parameters, θ_g, to make the generated samples more deceptive, reducing the discriminator's ability to distinguish them.
For fixed G,

$$V(G, D) = \int_x P_{data}(x)ln[D(x)] + P_g(x)ln[1 - D(G(z))]dx \tag{8}$$

Will be maximized when

$$D(x) = \frac{P_{data}(x)}{P_{data}(x) + P_g(x)} \tag{9}$$

If we fix $D(x) = \frac{P_{data}(x)}{P_{data}(x)+P_g(x)}$ and plug it into our value function, we find:

$$V = E_{x\ P_{data}}[ln(\frac{P_{data}(x)}{P_{data}(x) + P_g(x)})] + E_{x\ P_g}[ln(1 - \frac{P_{data}(x)}{P_{data}(x) + P_g(x)}] \tag{10}$$

When D is fixed, the generator minimizes this value, trying to make the generated data indistinguishable from real data, effectively fooling the discriminator.

To convince ourselves that the global minimum of the function V,occurs when $P_{data} = P_g$, observe that the formula for Jensen-Shannon divergence is similar to what we have:

$$JS(P_1||P_2) = E_{x\ P_1}[ln(\frac{P_1}{\frac{P_1+P_2}{2}})] + E_{x\ P_2}[ln(\frac{P_2}{\frac{P_1+P_2}{2}})] \tag{11}$$

Manipulate expression for $\min_G V$ to look like JS divergence.

$$\min_G V = E_{x\ P_{data}}[ln(\frac{P_{data}(x)}{P_{data}(x) + P_g(x)})] + E_{x\ P_g}[ln(\frac{P_g(x)}{P_{data}(x) + P_g(x)}] \tag{12}$$

$$\min_G V = E_{x\ P_{data}}[ln(\frac{P_{data}(x)}{\frac{P_{data}(x)+P_g(x)}{2}})] + E_{x\ P_g}[ln(\frac{P_g(x)}{\frac{P_{data}(x)+P_g(x)}{2}}] - 2ln2 \tag{13}$$

$$\min_{G} V = 2JS(P_{data}||P_g) - 2ln2 \tag{14}$$

The JS divergence cannot be negative, and it achieves its minimum of 0 when $P_{data} = P_g$. Thus, at the global minimum of the function V, $P_{data} = P_g$.

This min-max game represents the fundamental challenge in training GANs, where the generator and discriminator are in a continuous loop of adaptation against each other's strategies.

If we have extra information that we can apply to improve our model, we can make our GAN a Conditionl GAN which is formulated as follows. Suppose y is a vector containing extra information, typically in the form of a class label. Concatenate y with x and then feed it to the discriminator D. Then concatenate y with the noise z, and feed it to the generator G. The loss function can now be written:

$$V(G, D) = E_{x\ P_{data}}[ln(D(x|y))] + E_{z\ P_z}[ln(1 - D(G(z|y))] \tag{15}$$

This modification allows the GAN to generate and discriminate data conditioned on additional information, leading to more specialized and controlled generation processes.

III. CONDITIONAL GANS TO GENERATE CLOTHING IMAGES

For out first application, we use a cGAN to generate new images of clothing where the model is trained on the Fashion MNIST dataset. The generator takes a vector of random data as input and outputs a vector of fake data. The generator model includes 3 pairs of Linear and LeakyReLU layers, one more Linear layer, and a Tanh layer. The discriminator contains 11 layers with Conv2d, LeakyReLu, and BatchNorm2d activation functions and a twelfth layer with a sigmoid activation function. The Adam optimization algorithm is used to optimize both the generator and the discriminator. Additionally, we create a vector of random labels and concatenate it with the random data vector in the generator such that the fake data is now conditioned on these labels.

Figure 1 shows the clothing images generated by this method at different epochs. Observe that the images begin to look more like actual articles of clothing as the number of epochs increases. Figure 2 shows the generator and discriminator loss as a function of epoch. Note that the training loss converges to the same value as the number of epochs increases. Initially,

the real data and fake data distributions are far apart, so the discriminator is good at guessing which data is fake, so the discriminator loss is low. As the generator gets better at mimicking the real data, the generator loss decreases and the discriminator loss increases. A perfectly trained discriminator would have a loss of .5, indicating that the discriminator guesses correctly exactly half the time, meaning the generated data is very convincing. In this case, the losses converge to around 1. The fluctuations in the loss curves make sense since the discriminator and generator must keep reacting to one another.

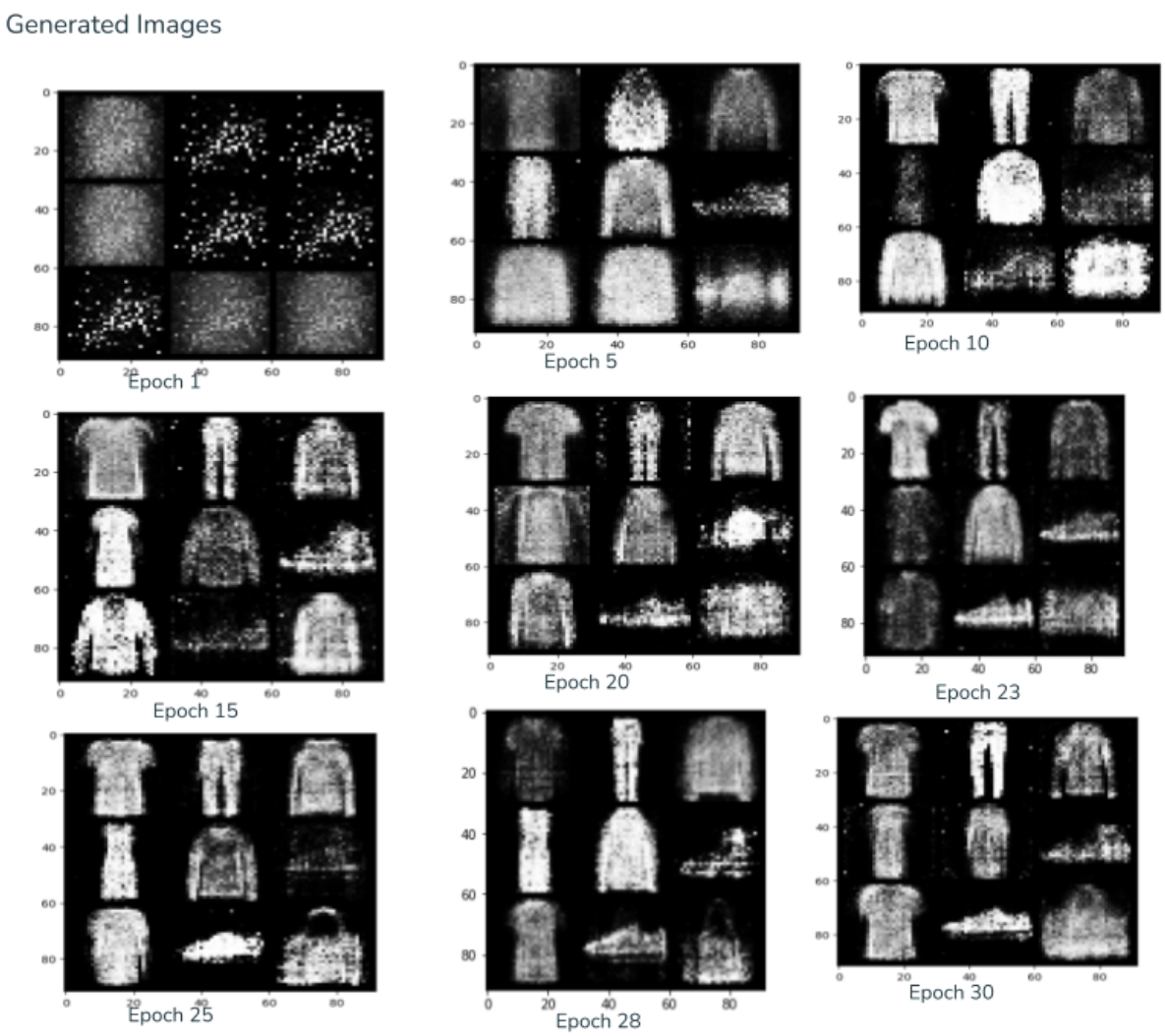

Figure 1: Generated images at different epochs

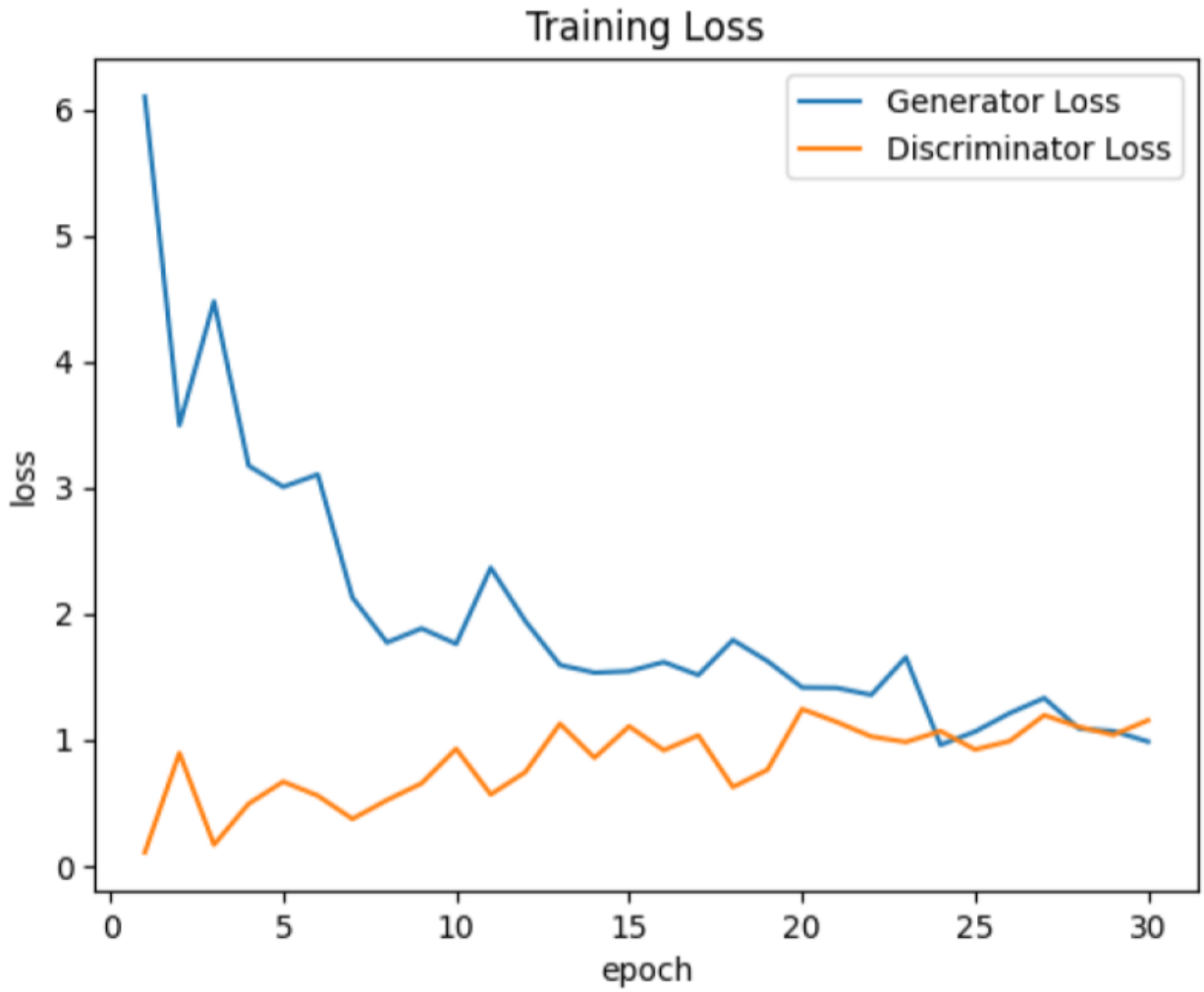

Figure 2: Generator (blue) and discriminator (orange) training loss

IV. STOCK PRICE MODELING USING GANS

The second application we consider is using a basic GAN to generate a model that can fit stock market price data from yahoo finance. The figures for this section come from the 2021 paper "Stock price prediction using Generative Adversarial Networks" [2]. We will discuss the effectiveness of the GAN models employed by the authors. The methodology they employed is as follows:

1. **Data Preparation:** Historical stock prices along with market indices and economic indicators are preprocessed to form training batches.

2. **Generator Architecture:** The generator, implemented using Gated Recurrent Units

(GRUs), processes input noise variables z to generate synthetic sequences of stock prices.

3. **Discriminator Architecture:** The discriminator, a Convolutional Neural Network (CNN), is trained to output the probability of the input data being real.

4. **Training Process:**

   (a) Alternate training of discriminator and generator, optimizing their respective loss functions L_D and L_G.

   (b) Use backpropagation for updating network weights.

   (c) Continue training until convergence, where generated data is indistinguishable from real data.

This methodology allows the Basic GAN to learn and generate new sequences of stock prices that reflect complex patterns in historical data, providing a novel tool for predicting future market behaviors. Figure 3 show the results of this model for test stock price data. Notice that the test model becomes less accurate after 2020. This could be caused by the COVID-19 pandemic since it led to uncertainty around stock prices as investors reacted to the changing market conditions. We can see that the accuracy in predicting the stock price in our model is the worst during that period.

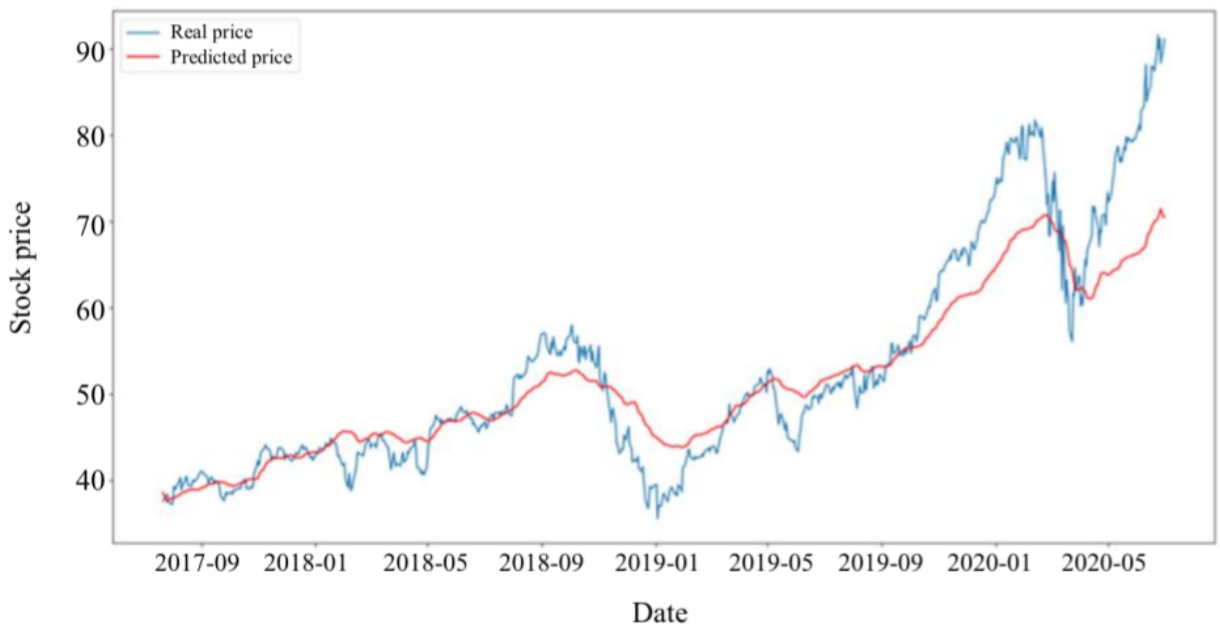

Figure 3: Model prediction using basic gan (red) plotted with testing data (blue) (Figure 7 in [2])

Let us compare the loss curves from training the basic GAN with the loss curves for the wGAN. wGAN methodology involves replacing traditional GAN loss functions with the Wasserstein distance, and the discriminator is trained to maximise the difference between the scores assigned to real and fake samples, rather than classify them (which is how basic GAN trains the discriminator).

Figure 4 shows the generator and discriminator loss for our basic GAN model. Observe that although the loss curves don't converge, the discriminator loss is around 1.4 which is not very far from the ideal value of .5. Figure 5 shows the training loss curves from the w-GAN. Observe that in this case the discriminator loss and generator loss curves do converge. The low generator loss for all epochs indicates that the fake data was similar to the real data at first, and remained thus throughout. The fact that the discriminator loss decreases indicates that the discriminator was initially very bad at distinguishing the real data from the fake, but got better as the model trained, and then got worse a little worse again as it dropped past .5.

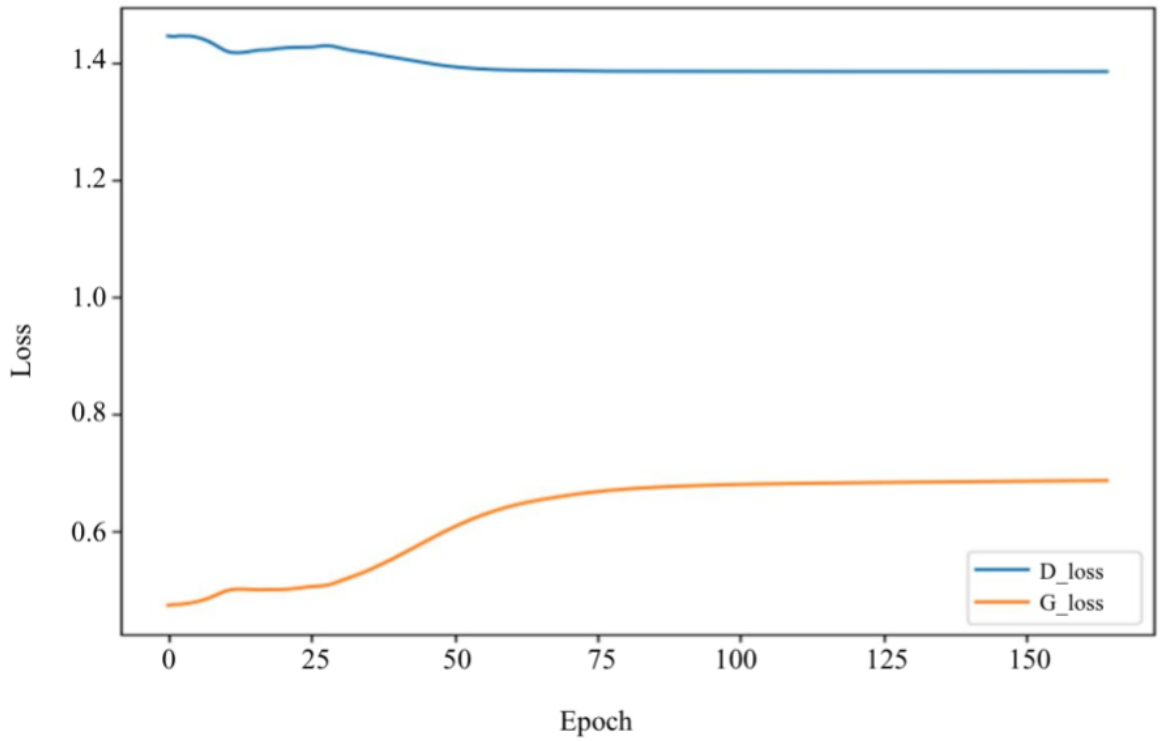

Figure 4: Discriminator loss (blue) and generator loss (orange) for basic GAN (Figure 6 in [2]).

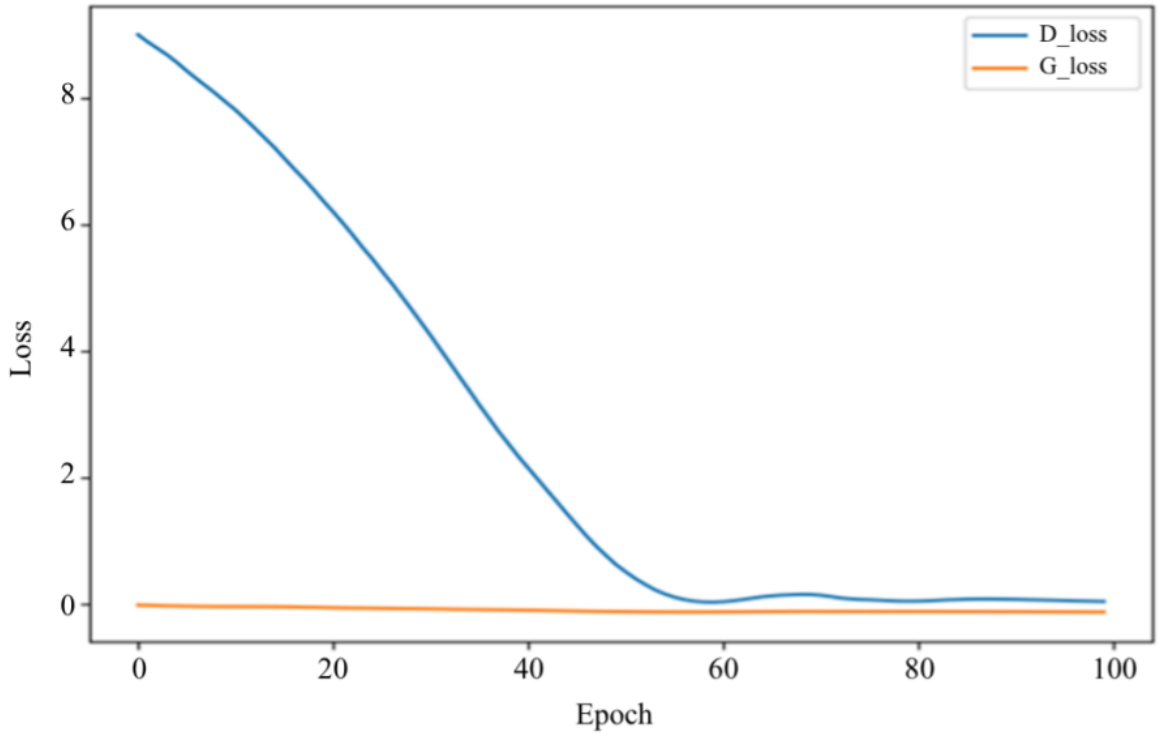

Figure 5: Discriminator loss (blue) and generator loss (orange) for wGAN (Figure 8 in [2]).

V. CONCLUSIONS

As we compare methodologies between our process and the paper in reference, we notice that they chose to proceed with WGAN rather than Basic GAN. Our methodology for stock price prediction used Basic GAN and for generating clothing images, CGAN. We can compare the convergence graphs of the discriminator and generator loss for each type of GAN. For CGAN, we do not have a stable gradient, however the overall trajectory (of best-fit) of the discriminator loss is decreasing. The generator loss is also unstable in manner yet steadily increases above zero. Training cGANs can be more challenging than training standard GANs due to the additional conditioning on labels, causing instability. Although there is eventual convergence, this explains the fluctuations in discriminator and generator losses that prevent them from converging to zero.

For our Basic GAN stock price model, we saw that after training the discriminator and generator loss, the functions became flat. This indicates stability in our model, but not full accuracy. When we compare our Basic GAN model with the 2021 paper by HungChun Lin,

Chen Chen, GaoFeng Huang and Amir Jafari using WGAN for their stock price model, we see that their discriminator and generator loss function converge to zero in a stable manner. add what it means to converge to zero

The reason why the comparison is stressing upon training loss graphs rather than the actual images produced is because it provides the in depth comparison of the 3 different types of GANs, and why Basic GAN is least effective in accuracy after training. WGANs methodology consists of replacing traditional GAN loss functions with the Wasserstein distance, and the discriminator is trained to maximise the difference between the scores assigned to real and fake samples, rather than classify them (which is how Basic GAN trains the discriminator). CGAN trains the functions based on conditioning both inputs with class labels, allowing for more targeted and accurate generation, which can be seen in not only the clothing images, but also the convergence model. The training epochs used in our Basic GAN application translate into showing that the discriminator initially has a higher loss than the generator, and even though it flattens, the gap remains. The referenced paper shows more effective learning in their WGAN model since it converges to zero leading to a point where the discriminator is not able to distinguish between real and generated data, and the generator is able to completely fool the discriminator. The generator's goal is to minimize this loss, meaning to create data so realistic that the discriminator mistakes it for real data The comparison between basic GAN and wGAN suggests that the discriminator loss of the wGAN converges more rapidly training. However, both models show potential for predicting stock prices, albeit with some limitations in handling unexpected events. Additionally, the use of cGANs for generating clothing images demonstrates promising results, indicating the versatility and effectiveness of GANs in various applications. Overall, while there are differences in performance between Basic GAN and WGAN-GP, both models contribute to the advancement of GAN technology and its applications in different domains.

---

[1] J. Schmidhuber, MIT Press/Bradford Books , 222–227 (1991).

[2] H. Lin, C. Chen, G. Huang, and A. Jafari, Journal of Computer Science **17**, 188 (2021).